Singly Linked List (Part A)

Learning Objectives

After completing this chapter, you will be able to:

- Define a Singly Linked List.
- Understand the structure of a node.
- Understand pointers in a linked list.
- Explain memory representation.
- Create nodes dynamically.
- Connect nodes together.
- Traverse and display a linked list.
- Write the first linked list program in C.

3.1 Introduction to Singly Linked List

A **Singly Linked List (SLL)** is the simplest type of linked list.

It is a **linear data structure** where each node contains:

- Data
- Address of the next node

Unlike arrays, elements are **not stored in consecutive memory locations**.

Each node knows only the address of the next node.

Definition

University Definition

A **Singly Linked List** is a linear data structure consisting of nodes, where each node contains data and a pointer to the next node in the sequence.

Simple Definition

A Singly Linked List is a collection of nodes connected one after another.

Tamil Explanation

Singly Linked List என்பது **ஒன்றுடன் ஒன்று இணைக்கப்பட்ட Node-களின் தொகுப்பாகும்.**

ஒவ்வொரு Node-லும்

- ஒரு Data
- அடுத்த Node-ன் Address

இருக்கும்.

3.2 Why is it called "Singly"?

The word **Single** means **one direction**.

Each node contains only **one link**.

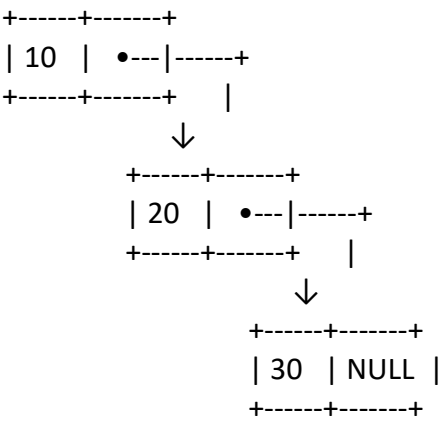
It points only to the **next node**.

It cannot move backwards.

Diagram

Head

↓



Traversal is

10

↓

20

↓

30

Only forward movement is possible.

Classroom Analogy

Imagine students standing in a queue.

Each student knows

only

who is standing in front.

They do not know

who is standing behind.

Exactly like a Singly Linked List.

3.3 Structure of a Node

Every linked list is built using **nodes**.

A node is the basic building block.

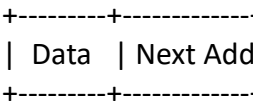
Every node contains

Data

+

Pointer

Diagram



Example

+------+------+
| 50 | •---|
+------+------+

Memory Representation
Suppose we store
10

20

30
The memory may look like
Address

100

250

400
100

+------+------+
| 10 | 250 |
+------+------+

250

+------+------+
| 20 | 400 |
+------+------+

400

+------+------+
| 30 | NULL |
+------+------+
Notice
Addresses are
NOT
continuous.

Important Observation
Arrays
100

104

108

112
Continuous Memory
Linked List
100

250

400

780
Random Memory
Pointers connect them.

3.4 Node Declaration in C
To create a node,
we use a structure.

```
#include<stdio.h>

struct Node
{
   int data;
   struct Node *next;
};
```

Explanation
struct Node
Creates a new data type.
int data;
Stores the value.
struct Node *next;
Stores the address of the next node.

Diagram
Node

+--------------------+

data

next

+--------------------+

Why do we write
struct Node *next;

instead of
int next;
Because
next must store
an address,
not a number.
Pointers store addresses.

Classroom Example
Suppose
Student A

↓

Student B

↓

Student C
Each student stores
the address
of the next student.

3.5 Creating the First Node
Memory for nodes is created dynamically.
Function used
malloc()
Syntax
pointer=(datatype *)malloc(size);
For Linked List
struct Node *newNode;

newNode=(struct Node *)malloc(sizeof(struct Node));

Explanation
sizeof(struct Node)
Calculates memory required for one node.
malloc()
Allocates memory.
Returns
address of new node.

Diagram
Before malloc
newNode

↓

NULL
After malloc
newNode

↓

Address 500

```
+------+------+
|      |      |
+------+------+
```

Assigning Data

```
newNode->data=10;
newNode->next=NULL;
```

Diagram

```
+------+------+
| 10   | NULL |
+------+------+
```

First Complete Program

```
#include<stdio.h>
#include<stdlib.h>

struct Node
{
   int data;
   struct Node *next;
};

int main()
{
   struct Node *head;

   head=(struct Node *)malloc(sizeof(struct Node));

   head->data=10;
   head->next=NULL;

   printf("Data = %d",head->data);

   return 0;
}
```

Output

```
Data = 10
```

Program Explanation
head
Stores
address of first node.

---

malloc()
Creates
one node.

---

head->data=10;
Stores value.

---

head->next=NULL;
There is no next node.

Memory After Execution
Head

↓

500

500

+------+------+
| 10 | NULL |
+------+------+

3.6 Creating Multiple Nodes
Suppose we want
10

20

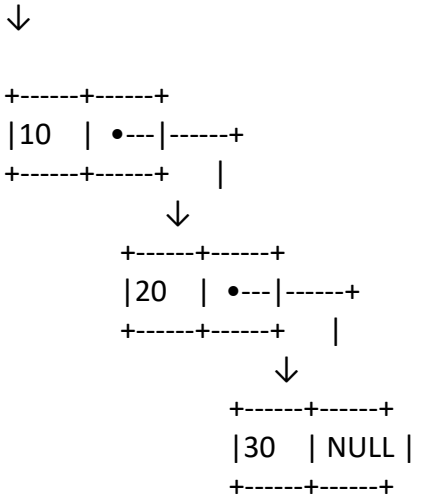
30
Program

```
struct Node *head;
struct Node *second;
struct Node *third;

head=(struct Node *)malloc(sizeof(struct Node));
second=(struct Node *)malloc(sizeof(struct Node));
third=(struct Node *)malloc(sizeof(struct Node));

head->data=10;
second->data=20;
third->data=30;
```

head->next=second;
second->next=third;
third->next=NULL;
Diagram
Head



3.7 Traversing a Singly Linked List
Traversal means
visiting every node one by one.
Algorithm
Step 1

Create temporary pointer

Step 2

Point temp=head

Step 3

Repeat until temp==NULL

Print temp->data

Move temp=temp->next

Stop

Program
#include<stdio.h>
#include<stdlib.h>

```
struct Node
{
   int data;
   struct Node *next;
};

int main()
{
   struct Node *head,*second,*third,*temp;

   head=(struct Node *)malloc(sizeof(struct Node));
   second=(struct Node *)malloc(sizeof(struct Node));
   third=(struct Node *)malloc(sizeof(struct Node));

   head->data=10;
   second->data=20;
   third->data=30;

   head->next=second;
   second->next=third;
   third->next=NULL;

   temp=head;

   while(temp!=NULL)
   {
      printf("%d ",temp->data);
      temp=temp->next;
   }

   return 0;
}
```

Output

10 20 30

Dry Run

Step	temp Points To	Output
1	10	10
2	20	10 20
3	30	10 20 30
4	NULL	Stop

Key Points to Remember

- A **Singly Linked List** is a collection of nodes connected by pointers.
- Each node contains **data** and a **pointer to the next node**.
- The **head** pointer stores the address of the first node.

- **malloc()** dynamically allocates memory for new nodes.
- The **last node's next pointer is always NULL**.
- Traversal is performed from **head** to **NULL**, visiting one node at a time.

3.8 Insertion in a Singly Linked List

What is Insertion?

Insertion is the process of adding a new node into an existing linked list.

Definition

Insertion is an operation that adds a new node to a linked list at a specified position.

Tamil Meaning

Insertion (செருகல்) என்பது இணைக்கப்பட்ட பட்டியலில் (Linked List) ஒரு புதிய Node-ஐ குறிப்பிட்ட இடத்தில் சேர்ப்பதாகும்.

Types of Insertion

There are three common insertion operations:

1. Insert at the Beginning
2. Insert at the End
3. Insert at a Specific Position

3.8.1 Insert at the Beginning

Suppose the existing list is:

Head
↓
10 → 20 → 30 → NULL

Insert **5** at the beginning.

Result

Head
↓
5 → 10 → 20 → 30 → NULL

Algorithm

1. Create a new node.
2. Store the data.
3. Make the new node point to the current head.
4. Move the head to the new node.

C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

struct Node *head = NULL;

void insertBeginning(int value)
{
```

```
    struct Node *newNode;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = value;

    newNode->next = head;

    head = newNode;
}
```

Explanation

Initially

Head

↓

10 → 20 → 30

After

newNode->next = head;

5

↓

10 → 20 → 30

After

head = newNode;

Head

↓

5 → 10 → 20 → 30

3.8.2 Insert at the End

Existing list

10 → 20 → 30 → NULL

Insert

40

Result

10 → 20 → 30 → 40 → NULL

Algorithm

1. Create a new node.
2. Set its next pointer to NULL.
3. Traverse to the last node.
4. Change the last node's next pointer.

C Function

```
void insertEnd(int value)
{
```

```
    struct Node *newNode;
    struct Node *temp;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->next = NULL;

    if(head == NULL)
    {
        head = newNode;
        return;
    }

    temp = head;

    while(temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->next = newNode;
}
```

Pointer Movement

Before

10 → 20 → 30 → NULL

After

10 → 20 → 30 → 40 → NULL

3.8.3 Insert at a Given Position

Example

Insert **25** at position **3**

Before

10 → 20 → 30 → 40

After

10 → 20 → 25 → 30 → 40

Algorithm

1. Create a new node.
2. Traverse to the node before the required position.
3. Point the new node to the next node.
4. Update the previous node's next pointer.

Pointer Illustration

Before

20 → 30

After

20 → 25 → 30

3.9 Deletion in a Singly Linked List

What is Deletion?

Deletion is the process of removing a node from the linked list.

Definition

Deletion removes an existing node and reconnects the remaining nodes.

Tamil Meaning

Deletion (நீக்குதல்) என்பது ஒரு Node-ஐ பட்டியலிலிருந்து அகற்றி, மீதமுள்ள Node-களை மீண்டும் இணைப்பதாகும்.

Types of Deletion

1. Delete from Beginning
2. Delete from End
3. Delete from Position

3.9.1 Delete from Beginning

Before

Head

↓

10 → 20 → 30

Delete first node.

After

Head

↓

20 → 30

Algorithm

1. Store the head node in a temporary pointer.
2. Move head to the next node.
3. Free the old first node.

C Function

```
void deleteBeginning()
{
   struct Node *temp;

   if(head == NULL)
   {
      printf("List is Empty");
      return;
   }

   temp = head;

   head = head->next;
```

```
    free(temp);
}
```

3.9.2 Delete from End

Before

10 → 20 → 30 → NULL

After

10 → 20 → NULL

Algorithm

1. Traverse to the last node.
2. Keep another pointer to the previous node.
3. Make previous node point to NULL.
4. Free the last node.

3.9.3 Delete from Position

Before

10 → 20 → 30 → 40

Delete

30

After

10 → 20 → 40

Only the pointer is changed.

3.10 Searching in a Linked List

Definition

Searching is the process of finding whether a given element exists in the linked list.

Algorithm

1. Start from head.
2. Compare each node.
3. If found, return its position.
4. Otherwise continue.
5. Stop at NULL.

C Function

```
void search(int key)
{
    struct Node *temp;

    int position = 1;

    temp = head;

    while(temp != NULL)
    {
        if(temp->data == key)
        {
            printf("Found at Position %d", position);
            return;
```

```
      }

      temp = temp->next;

      position++;
   }

   printf("Element Not Found");
}
```

Example
Searching
30
10 → 20 → 30 → 40
Result
Found at Position 3

3.11 Updating a Node

Updating means changing the value stored in an existing node.
Example
Before
10 → 20 → 30
Update
20 → 25
After
10 → 25 → 30

C Function

```
void update(int oldValue, int newValue)
{
   struct Node *temp = head;

   while(temp != NULL)
   {
      if(temp->data == oldValue)
      {
         temp->data = newValue;
         return;
      }

      temp = temp->next;
   }
}
```

3.12 Counting Nodes

Counting means finding the total number of nodes in the linked list.

Algorithm

1. Start from head.
2. Increase count.

3. Move to next node.
4. Repeat until NULL.

C Function

```
int countNodes()
{
   struct Node *temp = head;

   int count = 0;

   while(temp != NULL)
   {
      count++;

      temp = temp->next;
   }

   return count;
}
```

Example

10 → 20 → 30 → 40

Output

Total Nodes = 4

3.13 Traversal (Revision)

Traversal means visiting every node one by one.

```
void display()
{
   struct Node *temp = head;

   while(temp != NULL)
   {
      printf("%d ", temp->data);

      temp = temp->next;
   }
}
```

Output

10 20 30 40

Summary of Operations

Operation	Purpose
Create	Create a new linked list
Traverse	Display all nodes
Insert at Beginning	Add a node as the first node
Insert at End	Add a node as the last node
Insert at Position	Add a node at a specified position

Operation	Purpose
Delete from Beginning	Remove the first node
Delete from End	Remove the last node
Delete from Position	Remove a node from a given position
Search	Find an element
Update	Modify an existing element
Count	Count the total number of nodes

Real-World Applications

Music Playlist

- Insert a new song.
- Delete an existing song.
- Search for a song.
- Display all songs.

Student Attendance

- Add a new student.
- Remove a student.
- Search for a roll number.
- Update student details.

Browser History

- Add a new webpage.
- Delete old history.
- Traverse the browsing history.

Key Points to Remember

- Insertion and deletion are efficient because **only pointers are updated**.
- Unlike arrays, **elements are not shifted** during insertion or deletion.
- The **head pointer** always points to the first node.
- The **last node** always points to NULL.
- Every operation starts by traversing the list from the **head** unless the required node is already known.